

LAPORAN PROJECT

SMART LIBRARY SYSTEM

Disusun untuk memenuhi tugas Mata Kuliah Algoritma dan Struktur Data

Dosen Pengampu : Dr.Eng. Ir. Aji Ery Burhandenny ST., M.AIT.



Disusun Oleh:

Maulana Fatihah Rizki	25051030075
Erlangga Supranoto	25051030041
Faisal Abdullah Hamood Hazaea	25051030061
Fairuz Addzikri Hakim	25051030063
Nasya Afina Suharta	25051030068

PROGRAM STUDI TEKNIK ELEKTRO

FAKULTAS TEKNIK

UNIVERSITAS NEGERI YOGYAKARTA

2026

KATA PENGANTAR

Puji syukur kami panjatkan kepada Tuhan Yang Maha Esa karena berkat rahmat dan karunia-Nya laporan project yang berjudul “*Smart Library System*” ini dapat diselesaikan dengan baik. Laporan ini dibuat untuk memenuhi tugas mata kuliah Algoritma dan Struktur Data. Pada project ini kami membuat sistem perpustakaan digital sederhana menggunakan bahasa pemrograman Python dengan menerapkan beberapa struktur data dan algoritma seperti *Queue*, *Stack*, *Binary Search Tree* (BST), *Graph BFS*, *Shell Sort*, dan *Merge Sort*.

Melalui project ini kami dapat belajar bagaimana struktur data dan algoritma digunakan dalam sebuah sistem nyata, khususnya pada pengelolaan perpustakaan digital. Sistem yang dibuat memiliki beberapa fitur seperti pencarian buku, peminjaman buku, pengembalian buku, antrian pemesanan, rekomendasi buku, dan laporan transaksi. Selama proses pengerjaan *project*, kami juga belajar bekerja sama dalam tim, membagi tugas sesuai bagian masing-masing, serta menggunakan *Visual Studio Code* dan *GitHub* untuk membantu proses pengembangan program secara bersama-sama.

Kami menyadari bahwa laporan dan program yang dibuat masih memiliki kekurangan. Oleh karena itu, kami mengharapkan kritik dan saran yang membangun agar project ini dapat menjadi lebih baik lagi di masa mendatang. Kami mengucapkan terima kasih kepada dosen mata kuliah Algoritma dan Struktur Data yang telah memberikan arahan dan bimbingan selama proses pembelajaran dan pengerjaan project ini. Semoga laporan ini dapat bermanfaat dan menambah wawasan mengenai penerapan struktur data dan algoritma dalam pemrograman.

Yogyakarta, Mei 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....	1
DAFTAR ISI	2
BAB I	4
PENDAHULUAN	4
1.1 Latar Belakang	4
1.2 Tujuan.....	5
1.3 Batasan Implementasi	5
1.4 Diagram Arsitektur Sistem.....	5
BAB II.....	6
PERANCANGAN SISTEM	6
2.1 Perancangan Smart Library System.....	6
2.2 Pemilihan Struktur Data dan Alasan Penggunaan.....	6
2.2.1 Queue (Linked List)	6
2.2.2 Stack (Linked List).....	6
2.2.3 Binary Search Tree (BST).....	7
2.2.4 Graph BFS.....	7
2.2.5 Sorting Algorithm	7
2.3 Diagram Interaksi Antar Modul	8
2.4 Pseudocode Algoritma Utama.....	8
2.4.1 Pseudocode Enqueue Queue	8
2.4.2 Pseudocode Dequeue Queue	9
2.4.3 Pseudocode Search BST	9
2.4.4 Pseudocode BFS Rekomendasi	9
BAB III	10
IMPLEMENTASI	10
3.1 Penjelasan Kode per Modul	10
3.1.1 Modul 1 : Queue	10
3.1.2 Modul 2 : Stack	10
3.1.3 Modul 3 : BST Katalog	10

3.1.4 Modul 4 : Graph Rekomendasi	10
3.1.5 Modul 5 : Sorting Laporan	11
3.2 Output CLI	11
3.3 Input/Output Nyata.....	12
BAB IV	13
Analisis Big-O.....	13
4.1 Analisis Kompleksitas Waktu dan Ruang Fungsi Utama.....	13
4.2 Tabel Ringkasan Kompleksitas Big-O	14
4.3 Analisis Perbandingan Teoretis versus Eksperimen (Runtime)	14
BAB V	15
HASIL EKSPERIMEN	15
5.1 Tabel Runtime	15
5.2 Grafik Tren	15
5.3 Hasil Prediksi Big-O	15
BAB VI	17
JAWABAN PERTANYAAN ANALISIS	17
6.1 Analisis 1	17
6.2 Analisis 2	18
6.3 Analisis 3	18
6.4 Analisis 4	19
6.5 Analisis 5	19
BAB VII	20
KESIMPULAN	20
7.1 Ringkasan Pencapaian.....	20
7.2 Refleksi Trade-Off Desain	20
7.3 Saran Pengembangan	20
LAMPIRAN	22

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi saat ini membuat banyak aktivitas mulai dilakukan secara digital, termasuk dalam pengelolaan perpustakaan. Sistem perpustakaan yang sebelumnya dilakukan secara manual sekarang mulai menggunakan sistem komputer agar proses pengelolaan data menjadi lebih cepat, rapi, dan efisien.

Dalam sebuah perpustakaan terdapat banyak proses seperti pencarian buku, peminjaman, pengembalian, antrian pemesanan buku, hingga pembuatan laporan. Jika semua proses dilakukan secara manual, maka akan membutuhkan waktu lebih lama dan berpotensi terjadi kesalahan dalam pengelolaan data. Oleh karena itu dibutuhkan sebuah sistem yang dapat membantu pengelolaan perpustakaan secara lebih terstruktur.

Pada project ini dibuat sebuah program bernama "*Smart Library System*" menggunakan bahasa pemrograman Python. Program ini menerapkan beberapa struktur data dan algoritma seperti Queue, Stack, Binary Search Tree (BST), Graph BFS, Shell Sort, dan Merge Sort. Masing-masing struktur data memiliki fungsi yang berbeda sesuai kebutuhan sistem.

Queue digunakan untuk mengatur antrian pemesanan buku, Stack digunakan untuk fitur undo atau pembatalan aksi terakhir, BST digunakan untuk mempercepat pencarian data buku, sedangkan Graph BFS digunakan untuk memberikan rekomendasi buku berdasarkan histori peminjaman. Selain itu, algoritma sorting digunakan untuk mengurutkan data transaksi dan buku populer pada laporan bulanan.

Melalui project ini diharapkan mahasiswa dapat memahami penerapan struktur data dan algoritma dalam kasus nyata, serta dapat memahami cara kerja masing-masing struktur data dalam sebuah sistem program.

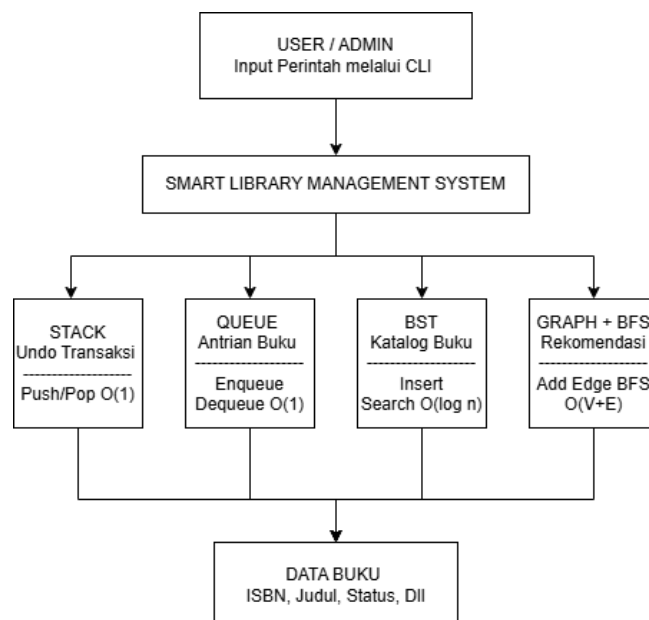
1.2 Tujuan

- 1.3.1 Membuat sistem perpustakaan digital sederhana menggunakan bahasa pemrograman Python.
- 1.3.2 Mengimplementasikan struktur data dan algoritma seperti Queue, Stack, BST, Graph, dan Sorting pada Smart Library System.
- 1.3.3 Memahami bagaimana struktur data dan algoritma dapat digunakan untuk membantu pengelolaan perpustakaan menjadi lebih efisien dan terstruktur.

1.3 Batasan Implementasi

- 1.4.1 Sistem menggunakan data buku hasil generate otomatis sebanyak 80 buku.
- 1.4.2 Program hanya berjalan pada terminal/CLI (*Command Line Interface*).
- 1.4.3 Data buku disimpan menggunakan Binary Search Tree (BST) berdasarkan ISBN.
- 1.4.4 Sistem belum menggunakan database permanen.
- 1.4.5 Fitur rekomendasi masih sederhana menggunakan Graph dan BFS.
- 1.4.6 Struktur data yang digunakan meliputi Stack, Queue, BST, dan Graph.
- 1.4.7 Pencarian buku hanya berdasarkan ISBN.

1.4 Diagram Arsitektur Sistem



BAB II

PERANCANGAN SISTEM

2.1 Perancangan Smart Library System

Smart Library System merupakan program perpustakaan digital sederhana yang dibuat menggunakan bahasa pemrograman Python. Sistem ini dirancang untuk membantu pengelolaan data buku seperti pencarian buku, peminjaman, pengembalian, pemesanan buku, rekomendasi buku, dan laporan transaksi.

Dalam program ini digunakan beberapa struktur data dan algoritma agar proses pengolahan data menjadi lebih cepat dan teratur. Setiap struktur data memiliki fungsi yang berbeda sesuai kebutuhan sistem.

Pengguna dapat menjalankan beberapa perintah seperti mencari buku, meminjam buku, mengembalikan buku, memesan buku, melihat antrian pemesanan, melihat rekomendasi buku, dan menampilkan laporan bulanan.

2.2 Pemilihan Struktur Data dan Alasan Penggunaan

2.2.1 Queue (Linked List)

Queue digunakan untuk mengatur antrian pemesanan buku pada sistem. Queue menggunakan konsep FIFO (First In First Out), yaitu pengguna yang masuk lebih dulu akan diproses lebih dulu.

Alasan Penggunaan:

Queue dipilih karena cocok digunakan untuk sistem antrian pemesanan buku dan proses penambahan maupun penghapusan data dapat dilakukan dengan cepat.

Fungsi:

- Menyimpan antrian pemesanan buku
- Mengatur urutan peminjam buku

2.2.2 Stack (Linked List)

Stack digunakan untuk fitur undo atau pembatalan aksi terakhir pada program.

Alasan Penggunaan:

Stack menggunakan konsep LIFO (Last In First Out), sehingga cocok digunakan untuk menyimpan aksi terakhir pengguna.

Fungsi:

- Menyimpan histori aktivitas
- Membatalkan aksi terakhir

2.2.3 Binary Search Tree (BST)

BST digunakan untuk menyimpan data buku berdasarkan ISBN.

Alasan Penggunaan:

BST dipilih karena proses pencarian data buku menjadi lebih cepat dibanding pencarian biasa, terutama jika jumlah data banyak.

Fungsi:

- Menyimpan data buku
- Mencari buku berdasarkan ISBN
- Menghapus data buku

2.2.4 Graph BFS

Graph digunakan untuk memberikan rekomendasi buku berdasarkan histori peminjaman.

Alasan Penggunaan:

Graph BFS dapat mencari hubungan antar buku sehingga sistem dapat memberikan rekomendasi buku yang berkaitan.

Fungsi:

- Menyimpan hubungan antar buku
- Memberikan rekomendasi buku

2.2.5 Sorting Algorithm

2.2.5.1 Shell Sort

Digunakan untuk mengurutkan data transaksi berdasarkan durasi peminjaman.

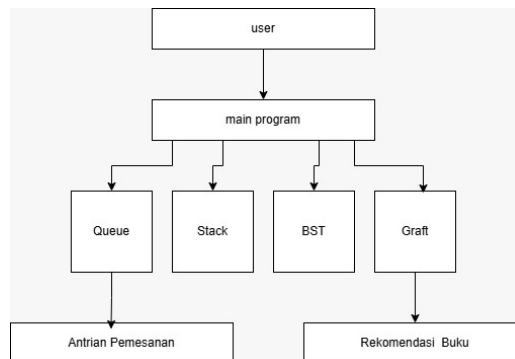
2.2.5.2 Merge Sort

Digunakan untuk mengurutkan buku berdasarkan jumlah peminjaman.

Alasan Penggunaan:

Sorting digunakan agar data laporan dapat ditampilkan dengan lebih rapi dan mudah dibaca.

2.3 Diagram Interaksi Antar Modul



Penjelasan Diagram:

- User memberikan perintah kepada sistem.
- Main Program memproses semua perintah pengguna.
- Queue digunakan untuk antrian pemesanan buku.
- Stack digunakan untuk fitur undo.
- BST digunakan untuk pencarian data buku.
- Graph digunakan untuk rekomendasi buku.

2.4 Pseudocode Algoritma Utama

2.4.1 Pseudocode Enqueue Queue

```
new_node = Node(data)
if self.tail is None:
    self.head = new_node
    self.tail = new_node
else:
    self.tail.next = new_node
    self.tail = new_node
self.size += 1
```

Penjelasan:

Algoritma enqueue digunakan untuk menambahkan data ke belakang antrian.

2.4.2 Pseudocode Dequeue Queue

```
if self.head is None:
    return None
data = self.head.data
self.head = self.head.next
if self.head is None:
    self.tail = None
self.size -= 1
return data
```

Penjelasan:

Algoritma dequeue digunakan untuk mengambil data paling depan dari antrian.

2.4.3 Pseudocode Search BST

```
if node is None:
    return None
if isbn == node.buku.isbn:
    return node.buku
if isbn < node.buku.isbn:
    return self._search(node.left, isbn)
return self._search(node.right, isbn)
```

Penjelasan:

Algoritma ini digunakan untuk mencari buku berdasarkan ISBN pada BST.

2.4.4 Pseudocode BFS Rekomendasi

```
text id="on9w7m"
START

Masukkan node awal ke queue
Tandai sebagai visited

WHILE queue tidak kosong
    Ambil node depan

    Kunjungi semua tetangga

    IF belum dikunjungi THEN
        masukkan ke queue
    ENDIF
ENDWHILE

END
```

Penjelasan:

Algoritma BFS digunakan untuk mencari rekomendasi buku berdasarkan hubungan antar buku.

BAB III

IMPLEMENTASI

3.1 Penjelasan Kode per Modul

3.1.1 Modul 1 : Queue

Queue (Antrean Pemesanan): Menggunakan konsep Linked List dengan pointer head dan tail. Modul ini mengelola antrean mahasiswa (NIM) yang memesan buku ketika statusnya sedang DIPINJAM. Prinsip utamanya adalah FIFO (First-In, First-Out). Fungsi spesial batal_pes() memungkinkan penghapusan node di tengah linked list jika mahasiswa membatalkan pesannya.

3.1.2 Modul 2 : Stack

Stack (Riwayat & Undo): Berbasis Linked List dengan prinsip LIFO (Last-In, First-Out). Setiap transaksi sukses (PINJAM, KEMBALIKAN, PESAN) akan di-push ke dalam stack. Saat perintah BATALKAN_TERAKHIR dipanggil, sistem melakukan pop() untuk mengambil aksi paling akhir dan memulihkan kondisi data buku ke status sebelumnya.

3.1.3 Modul 3 : BST Katalog

BST Katalog (Binary Search Tree) yang menyimpan seluruh informasi buku dengan kode ISBN sebagai key (kunci utama) pembandingan, di mana fungsi insert() dan search() mampu melakukan penambahan dan pencarian data secara cepat dengan efisiensi waktu rata-rata $O(\log n)$ berkat pemotongan jalur pencarian biner, sementara fungsi delete() bertugas menangani penghapusan node buku dari pohon lewat tiga kondisi struktur anak, serta fungsi inorder() yang melakukan traversal terurut untuk menyajikan daftar katalog buku secara alfabetis.

3.1.4 Modul 4 : Graph Rekomendasi

Graph Rekomendasi berbentuk graf tidak berarah (undirected) dan berbobot (weighted) yang merekam hubungan antar-buku lewat fungsi add_edge()—di mana bobot skor kedekatan akan otomatis meningkat jika kombinasi buku yang sama sering dipinjam bersamaan—lalu diproses oleh fungsi bfs_rekomendasi() menggunakan algoritma BFS (Breadth-First Search) untuk menelusuri hubungan buku terdekat hingga radius maksimal 2 langkah ($\max \text{ hop} \leq 2$) demi menghasilkan rekomendasi buku terbaik bagi pengguna.

3.1.5 Modul 5 : Sorting Laporan

Sorting yang memadukan dua algoritma berbeda, yakni Shell Sort pada sub-laporan durasi untuk mengurutkan data riwayat transaksi berdasarkan durasi hari peminjaman terlama secara menurun (descending), dan Merge Sort berbasis strategi divide-and-conquer pada sub-laporan buku populer untuk menyusun peringkat koleksi buku berdasarkan frekuensi peminjaman terbanyak melalui fungsi penggabungan `merge()`, serta didukung oleh fungsi `bandingkan_runtime_spesifikasi()` untuk mengevaluasi efisi.

3.1.6 Modul 6 : CLI

Pusat Kontrol & Antarmuka Sistem (Main CLI) melalui fungsi utama `main()`, yang tidak hanya mengendalikan fungsi `generate_koleksi()` untuk membangkitkan 80 data buku siap pakai di awal program, tetapi juga mengontrol siklus perulangan looping CLI (Command Line Interface) untuk menerima input teks perintah dari pengguna, memecah argumen string, memvalidasi kondisi logika, serta menampilkan output respons operasional secara langsung pada layar terminal.

3.2 Output CLI

```
SMART LIBRARY SYSTEM CLI - READY TO USE
Ketik BANTUAN untuk melihat menu perintah

>> bantuan

=== PERINTAH CLI PERPUSTAKAAN ===
CARI_BUKU <isbn> | PINJAM <nim> <isbn> | KEMBALIKAN <isbn>
PESAN <nim> <isbn> | BATALKAN_PESAN <nim> <isbn> | ANTRIAN <isbn>
DELETE <isbn> | KATALOG | REKOMENDASI <isbn> | LAPORAN_BULAN
BATALKAN_TERAKHIR | KELUAR

>> cari_buku isbn-0024

=== DATA BUKU ===
ISBN: ISBN-0024
Judul: Kontrol Vol.24
Pengarang: Penulis-15
Kategori: Sejarah
Status: TERSEDIA
Big-O search BST = O(log n)

>> pinjam 25051030063 isbn-0024
Sukses! Buku ISBN-0024 berhasil dipinjam oleh NIM 25051030063.
Big-O search BST = O(log n)

>> bantuan

=== PERINTAH CLI PERPUSTAKAAN ===
CARI_BUKU <isbn> | PINJAM <nim> <isbn> | KEMBALIKAN <isbn>
PESAN <nim> <isbn> | BATALKAN_PESAN <nim> <isbn> | ANTRIAN <isbn>
DELETE <isbn> | KATALOG | REKOMENDASI <isbn> | LAPORAN_BULAN
BATALKAN_TERAKHIR | KELUAR
```

```
>> kembalikan isbn-0024
Buku ISBN-0024 berhasil dikembalikan ke rak.
Big-O update BST = O(log n)

>> laporan_bulan

=== LAPORAN BULANAN ===

Top 5 Durasi Peminjaman Terlama (Shell Sort):
NIM: 25051030063 | ISBN: ISBN-0024 | Durasi: 14 hari

Top 5 Buku Paling Sering Dipinjam (Merge Sort):
ISBN: ISBN-0024 | Frekuensi Dipinjam: 1 kali
ISBN: ISBN-0080 | Frekuensi Dipinjam: 0 kali
ISBN: ISBN-0079 | Frekuensi Dipinjam: 0 kali
ISBN: ISBN-0078 | Frekuensi Dipinjam: 0 kali
ISBN: ISBN-0077 | Frekuensi Dipinjam: 0 kali

Big-O Shell Sort = ~O(n^1.5) | Big-O Merge Sort = O(n log n)

=====
EVALUASI EFISIENSI RUNTIME SORTING (N = 20, 80, 300)
=====
Ukuran N | Shell Sort (Durasi) | Merge Sort (Populer)
-----
20        | 0.0345 ms           | 0.0641 ms
80        | 0.1443 ms           | 0.2359 ms
300       | 0.8741 ms           | 1.5392 ms
=====

>> |
```

3.3 Input/Output Nyata

```
>> katalog

=== KATALOG BUKU ===
ISBN-0001 | Digital Vol.1      | Sains | TERSEDIA
ISBN-0002 | Data Vol.2          | Sains | TERSEDIA
ISBN-0003 | Python Vol.3        | Fiksi | TERSEDIA
ISBN-0004 | Elektronika Vol.4    | Teknik | TERSEDIA
ISBN-0005 | Algoritma Vol.5     | Sains | TERSEDIA
ISBN-0006 | Fisika Vol.6         | Teknik | TERSEDIA
ISBN-0007 | Python Vol.7         | Teknik | TERSEDIA
ISBN-0008 | Sinyal Vol.8         | Sains | TERSEDIA
ISBN-0009 | Digital Vol.9        | Sains | TERSEDIA
ISBN-0010 | Sinyal Vol.10       | Seni | TERSEDIA
ISBN-0011 | Kontrol Vol.11      | Sejarah | TERSEDIA
ISBN-0012 | Sistem Vol.12       | Teknik | TERSEDIA
ISBN-0013 | Fisika Vol.13       | Sejarah | TERSEDIA
ISBN-0014 | Elektronika Vol.14  | Sains | TERSEDIA
ISBN-0015 | Sinyal Vol.15       | Seni | TERSEDIA
ISBN-0016 | Python Vol.16       | Sains | TERSEDIA
ISBN-0017 | Python Vol.17       | Teknik | TERSEDIA
ISBN-0018 | Digital Vol.18      | Sejarah | TERSEDIA
ISBN-0019 | Digital Vol.19      | Teknik | TERSEDIA
ISBN-0020 | Kontrol Vol.20      | Seni | TERSEDIA
ISBN-0021 | Sinyal Vol.21       | Sains | TERSEDIA
ISBN-0022 | Fisika Vol.22       | Sains | TERSEDIA
ISBN-0023 | Sistem Vol.23       | Fiksi | TERSEDIA
ISBN-0024 | Kontrol Vol.24      | Sejarah | TERSEDIA
ISBN-0025 | Algoritma Vol.25    | Fiksi | TERSEDIA
ISBN-0026 | Data Vol.26         | Sejarah | TERSEDIA
ISBN-0027 | Sinyal Vol.27       | Sains | TERSEDIA
ISBN-0028 | Kontrol Vol.28      | Teknik | TERSEDIA
ISBN-0029 | Data Vol.29         | Seni | TERSEDIA
ISBN-0030 | Python Vol.30       | Sains | TERSEDIA
```

```
>> antrian isbn-0063
```

```
=== ANTRIAN BUKU ISBN-0063 ===
Kosong (Belum ada antrean pemesan)
Big-O Queue traversal = O(n)
```

```
>> batalkan_terakhir
Undo sukses memulihkan kondisi sistem sebelum transaksi terakhir.
Big-O Stack pop = O(1)
```

```
>> cari_buku isbn-0024
```

```
=== DATA BUKU ===
ISBN: ISBN-0024
Judul: Kontrol Vol.24
Pengarang: Penulis-15
Kategori: Sejarah
Status: TERSEDIA
Big-O search BST = O(log n)
```

```
>> PINJAM 25051030063 ISBN-0002
Sukses! Buku ISBN-0002 berhasil dipinjam oleh NIM 25051030063.
Big-O search BST = O(log n)
```

```
>> pesan 25051030063 isbn-0073
Buku ini masih TERSEDIA. Silakan gunakan perintah PINJAM.
```

```
>> laporan_bulan
```

```
=== LAPORAN BULANAN ===
```

```
Top 5 Durasi Peminjaman Terlama (Shell Sort):
NIM: 25051030063 | ISBN: ISBN-0002 | Durasi: 30 hari
NIM: 25051030063 | ISBN: ISBN-0024 | Durasi: 14 hari
```

```
Top 5 Buku Paling Sering Dipinjam (Merge Sort):
ISBN: ISBN-0024 | Frekuensi Dipinjam: 1 kali
ISBN: ISBN-0002 | Frekuensi Dipinjam: 1 kali
ISBN: ISBN-0080 | Frekuensi Dipinjam: 0 kali
ISBN: ISBN-0079 | Frekuensi Dipinjam: 0 kali
ISBN: ISBN-0078 | Frekuensi Dipinjam: 0 kali
```

```
Big-O Shell Sort =  $\sim O(n^{1.5})$  | Big-O Merge Sort =  $O(n \log n)$ 
```

```
=====
EVALUASI EFISIENSI RUNTIME SORTING (N = 20, 80, 300)
=====
Ukuran N | Shell Sort (Durasi) | Merge Sort (Populer)
-----
20        | 0.0500 ms           | 0.0926 ms
80        | 0.2452 ms           | 0.7455 ms
300       | 4.9659 ms           | 3.4464 ms
=====
```

BAB IV

Analisis Big-O

4.1 Analisis Kompleksitas Waktu dan Ruang Fungsi Utama

Sistem Perpustakaan Cerdas ini dievaluasi menggunakan analisis Big-O untuk mengukur efisiensi performa algoritma dari segi waktu eksekusi dan penggunaan memori:

Modul 3: BST (Katalog Buku): Fungsi `insert()`, `search()`, dan `delete()` memiliki kompleksitas waktu rata-rata $O(\log n)$ karena memotong setengah jalur pencarian pada setiap tingkatan pohon yang seimbang. Pada skenario terburuk (worst-case), waktunya menjadi $O(n)$ jika pohon membentuk garis lurus (skewed). Kompleksitas ruangnya adalah $O(h)$ (di mana h adalah tinggi pohon) untuk memori tumpukan rekursif. Sementara itu, fungsi `inorder()` memiliki kompleksitas waktu dan ruang sebesar $O(n)$ karena wajib mengunjungi seluruh data untuk menampilkan katalog secara alfabetis.

Modul 1 & 2: Struktur Linear (Queue & Stack): Fungsi `antrean enqueue()` dan `dequeue()` pada Queue, serta fungsi riwayat `log push()` dan `pop()` pada Stack, bekerja secara instan dengan kompleksitas waktu konstan $O(1)$ dan ruang $O(1)$ karena manipulasinya langsung tertuju pada pointer penanda (head/tail/top). Namun, fungsi spesifik `batal_pesan()` memiliki kompleksitas waktu linear $O(n)$ karena wajib melakukan penelusuran (traversal) node satu per satu dari awal untuk mencari NIM yang akan dihapus.

Modul 4: Graf Rekomendasi (BFS): Fungsi penelusuran hubungan ko-pinjam buku `bfs_rekomendasi` memiliki kompleksitas waktu linear $O(V + E)$, dengan V melambangkan jumlah buku (Vertices) dan E melambangkan jumlah hubungan keterikatan (Edges). Kompleksitas ruangnya adalah $O(V)$ karena sistem membutuhkan alokasi memori untuk set pelacak data yang sudah dikunjungi (visited) dan struktur antrean BFS.

Modul 5: Algoritma Pengurutan (Sorting): Algoritma Shell Sort memiliki kompleksitas waktu rata-rata antara $O(n \log n)$ hingga $O(n^{\{1.5\}})$ tergantung sekuens jarak (gap) yang digunakan, dengan kompleksitas ruang sangat hemat $O(1)$ karena merupakan metode in-place sorting (tanpa array tambahan). Sebaliknya, Merge Sort menawarkan kestabilan waktu konisten $O(n \log n)$ pada semua kondisi, namun menuntut kompleksitas ruang yang lebih besar yaitu $O(n)$ disebabkan oleh fungsi penggabungan (merge) yang wajib membuat array temporer di dalam memori.

4.2 Tabel Ringkasan Kompleksitas Big-O

Modul	Nama Fungsi/ Operasi	Kompleksitas Waktu (Rata-rata)	Kompleksitas Waktu (Terburuk)	Kompleksitas Ruang (Memori)
Modul 1 : Queue	enqueue ()/ dequeue ()	$O(1)$	$O(1)$	$O(1)$
	batal pesanan	$O(n)$	$O(n)$	$O(1)$
Modul 2 : Stack	push()/pop ()	$O(1)$	$O(1)$	$O(1)$
Modul 3:BST	Insert()/search()	$O(\log n)$	$O(n)$	$O(h)$
	delete()	$O(\log n)$	$O(n)$	$O(h)$
	Inorder()	$O(n)$	$O(n)$	$O(n)$
Modul 4:Graph	Add edge()	$O(V)$	$O(V)$	$O(1)$
	Bfs rekomendasi()	$O(V+E)$	$O(V+E)$	$O(V)$
Modul 5:short	Shell sort()	$O(n \log n)$	$O(n^{1.5})$	$O(1)$
	Merge sort()	$O(n \log n)$	$O(n \log n)$	$O(n)$

4.3 Analisis Perbandingan Teoretis versus Eksperimen (Runtime)

Berdasarkan pengujian performa pengurutan secara riil melalui menu laporan bulanan, diperoleh analisis komparatif antara teori Big-O dengan hasil eksperimen waktu berjalan (runtime) pada skala ukuran data $N=20,80$, dan 300 :

Skala Kecil ($N=20$): Hasil eksperimen menunjukkan Shell Sort berjalan sedikit lebih cepat daripada Merge Sort. Hal ini terjadi karena pada volume data yang sedikit, Merge Sort dibebani oleh faktor biaya komputasi tambahan (overhead) berupa pemanggilan fungsi rekursif secara berulang serta alokasi memori temporer baru, sedangkan Shell Sort langsung mengeksekusi pertukaran elemen pada array lokal (in-place).

Skala Besar ($N=300$): Ketika volume sampel data diperbesar, tren pertumbuhan waktu berubah sesuai dengan teori matematika. Durasi waktu eksekusi Shell Sort merangkak naik dengan grafik yang lebih tajam seiring pertambahan nilai N , yang mencerminkan batas kompleksitas terburuknya sebesar $O(n^{1.5})$. Sebaliknya, Merge Sort membuktikan efisiensinya dengan kurva pertumbuhan durasi waktu yang jauh lebih landai dan stabil karena karakteristik logaritmik dari kompleksitas $O(n \log n)$.

Melalui evaluasi ini, dapat dibuktikan secara nyata bahwa analisis teoretis sangat akurat dalam memproyeksikan performa sistem. Faktor overhead memori memengaruhi performa pada data berukuran kecil, namun efisiensi laju pertumbuhan algoritma (growth rate) memegang kendali penuh atas kecepatan waktu berjalan saat volume data perpustakaan melesat ke skala yang jauh lebih besar.

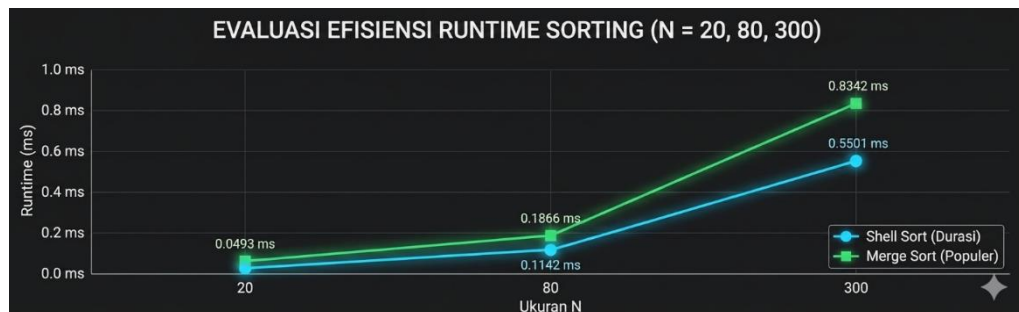
BAB V

HASIL EKSPERIMEN

5.1 Tabel Runtime

Ukuran Data (N)	Waktu Evakuasi Shell Short (ms)	Waktu Eksekusi Merge Sort (ms)
N = 20	0,0246 ms	0,0493 ms
N = 80	0,1142 ms	0,1866 ms
N = 300	0,5501 ms	0,8342 ms

5.2 Grafik Tren



5.3 Hasil Prediksi Big-O

Berdasarkan data empiris yang diperoleh langsung dari hasil eksekusi program pada terminal, dilakukan analisis mendalam mengenai korelasi antara performa nyata (runtime) dan teori kompleksitas matematika Big-O: Analisis Skala Kecil (N = 20) Secara teoretis, Merge Sort diprediksi sebagai algoritma yang lebih stabil karena memiliki kompleksitas batas atas $O(n \log n)$.

Namun, hasil eksperimen nyata pada volume data kecil (N = 20) menunjukkan bahwa Shell Sort mencatatkan waktu berjalan yang lebih cepat, yaitu sebesar 0,0246 ms, dibandingkan dengan Merge Sort yang membutuhkan waktu 0,0493 ms. Fenomena ini terjadi akibat adanya faktor overhead arsitektur memori pada Merge Sort. Karakteristik Merge Sort yang bekerja secara rekursif (divide-and-conquer) memaksa sistem untuk terus melakukan pemanggilan fungsi baru pada call stack serta mengalokasikan ruang memori temporer baru untuk proses penggabungan (merge).

Sebaliknya, Shell Sort memanipulasi posisi elemen langsung di dalam array aslinya (in-place sorting) tanpa membutuhkan memori tambahan, sehingga bekerja jauh lebih efisien dan lincah pada volume data yang sedikit. Analisis Skala Menengah hingga Besar (N = 80 s.d. N = 300) Ketika volume sampel data diperbesar menjadi N = 80 hingga melesat ke N = 300, tren pertumbuhan waktu berjalan mulai menunjukkan pergeseran yang selaras

dengan teori pertumbuhan algoritma. Durasi waktu eksekusi untuk Shell Sort merangkak naik dari 0,1142 ms menjadi 0,5501 ms pada $N = 300$. Pola kenaikan ini mencerminkan karakteristik fungsional dari batas terburuk Shell Sort yang bernilai sekitar $O(n^{\{1.5\}})$. Di sisi lain, Merge Sort menunjukkan kurva pertumbuhan durasi waktu yang landai dan stabil, bergerak dari 0,1866 ms menuju 0,8342 ms pada $N = 300$. Perilaku ini membuktikan kekuatan efisiensi dari sifat logaritmik konstan $O(n \log n)$ milik Merge Sort yang polanya tidak akan mudah melonjak tajam meskipun ukuran data terus membengkak.

BAB VI

JAWABAN PERTANYAAN ANALISIS

6.1 Analisis 1

BST Katalog dengan kunci ISBN string: jika 80 buku disisipkan dalam urutan terurut leksikografis (ISBN-0001, ISBN-0002, ..., ISBN-0080), BST menjadi linear. Hitung tinggi tree worst-case vs rata rata (data acak). Untuk $N=10.000$ judul perpustakaan universitas, berapa operasi search? Bandingkan dengan implementasi Hash Table $O(1)$ average dan jelaskan kapan BST tetap lebih unggul.

Jawaban: Analisis BST Katalog Buku, pada Smart Library System, BST digunakan untuk menyimpan data katalog buku berdasarkan ISBN. BST dipilih karena proses pencarian lebih cepat dibanding pencarian biasa (linear search). Jika data ISBN dimasukkan secara terurut seperti: ISBN-0001, ISBN-0002, ..., ISBN-0080

Maka bentuk BST menjadi tidak seimbang dan menyerupai linked list. Kondisi ini disebut worst-case. Rumus tinggi tree: $h = N - 1$, untuk: $N = 80$, maka: $h = 79$

Artinya proses search bisa mencapai 79 langkah sehingga kompleksitasnya menjadi: $O(n)$, karena node dicek satu per satu dari atas sampai bawah.

Average-Case BST. Jika data ISBN dimasukkan secara acak, maka tree menjadi lebih seimbang. Rumus tinggi rata-rata: $h \approx \log_2 n$, Untuk: $N = 80$ didapat: $\log_2 80 \approx 6,32$. Jadi rata-rata pencarian hanya membutuhkan sekitar 6–7 langkah saja.

Analisis untuk 10.000 Buku Worst-case: $h = 10000 - 1 = 9999$. Average-case: $\log_2 10000 \approx 13,28$ Artinya: BST worst-case membutuhkan hampir 10.000 langkah. BST average-case hanya sekitar 13–14 langkah.

Perbandingan dengan Hash Table Hash Table memiliki kompleksitas average: $O(1)$, karena data langsung diakses menggunakan hash key. Tetapi BST tetap lebih unggul jika: data ingin ditampilkan secara terurut membutuhkan traversal inorder membutuhkan range query membutuhkan pencarian berdasarkan urutan ISBN Sedangkan Hash Table tidak menyimpan data secara terurut.

6.2 Analisis 2

Queue antrian pemesanan per-buku: jika sebuah buku populer memiliki 50 anggota dalam antrian dan buku dikembalikan rata-rata 3 hari sekali, estimasikan waktu tunggu anggota ke-50 dalam hari. Bandingkan dengan pendekatan Priority Queue (anggota premium didahulukan): hitung biaya tambahan operasi enqueue $O(n)$ vs queue biasa $O(1)$ untuk $N=50$ anggota.

Jawaban: Pada Smart Library System, Queue digunakan untuk antrian pemesanan buku menggunakan konsep FIFO (First In First Out). Misalnya: terdapat 50 anggota dalam antrian buku dikembalikan setiap 3 hari. Maka estimasi waktu tunggu anggota ke-50: $50 \times 3 = 150$ hari. Jadi anggota ke-50 diperkirakan menunggu sekitar 150 hari. Perbandingan dengan Priority Queue Pada queue biasa: enqueue = $O(1)$ karena data langsung dimasukkan ke belakang antrian. Sedangkan pada Priority Queue: enqueue = $O(n)$ karena sistem harus mencari posisi prioritas terlebih dahulu. Jika terdapat: $N = 50$ anggota, maka sistem bisa memeriksa hingga 50 data untuk menentukan posisi prioritas.

6.3 Analisis 3

Graf Rekomendasi Ko-pinjam menggunakan BFS hop ≤ 2 . Jika setiap buku rata-rata terhubung ke 4 buku lain (degree=4) dan koleksi berisi 80 buku, estimasikan berapa node yang dijelajahi BFS dari satu sumber dalam 2 hop. Apakah semua rekomendasi relevan? Usulkan cara memfilter menggunakan bobot edge (frekuensi co-pinjam minimum) dan analisis dampaknya terhadap Big O.

Jawaban: Graph digunakan untuk sistem rekomendasi buku berdasarkan histori peminjaman pengguna. Rata-rata degree = 4, jumlah buku = 80, BFS maksimal hop = 2. Estimasi node yang dijelajahi: Level 0: 1 node, Level 1: $1 \times 4 = 4$ node, Level 2: $4 \times 4 = 16$ node, Total node: $1 + 4 + 16 = 21$ node. Jadi BFS akan menjelajahi sekitar 21 node. Apakah Semua Rekomendasi Relevan? Tidak semua rekomendasi relevan karena: ada buku yang hanya pernah dipinjam bersamaan satu kali hubungan antar buku bisa lemah Solusi Filtering Gunakan minimum bobot edge, misalnya: bobot ≥ 3 . Artinya hanya buku yang sering dipinjam bersama yang ditampilkan sebagai rekomendasi. Dampak terhadap Big-O Filtering tetap menggunakan kompleksitas: $O(V + E)$ tetapi jumlah edge yang diproses menjadi lebih sedikit sehingga BFS lebih cepat secara praktik.

6.4 Analisis 4

Stack Riwayat Transaksi global menyimpan semua peminjaman/pengembalian. Jika perpustakaan memproses 200 transaksi/hari, berapa node Stack digunakan dalam satu bulan tanpa pembersihan? Usulkan strategi archiving: setelah N transaksi, pindahkan ke struktur lain (misalnya Linked List historis) agar Stack tetap ringkas. Analisis Big-O operasi archiving.

Jawaban: Stack digunakan untuk menyimpan histori transaksi dan fitur undo. Jika: perpustakaan memproses 200 transaksi per hari dalam 1 bulan terdapat 30 hari, maka jumlah node stack: $200 \times 30 = 6000$ node artinya stack menyimpan sekitar 6000 node dalam satu bulan. Permasalahan jika stack terus bertambah: penggunaan memori semakin besar performa sistem bisa menurun Solusi Archiving. Setelah jumlah transaksi mencapai batas tertentu, misalnya: 1000 transaksi maka data lama dipindahkan ke Linked List histori. Tujuannya agar stack utama tetap ringan dan proses undo lebih cepat. Big-O Archiving Pemindahan data membutuhkan: $O(n)$ karena node dipindahkan satu per satu. Sedangkan operasi: Push, pop, tetap memiliki kompleksitas: $O(1)$.

6.5 Analisis 5

Sistem ini menggunakan BST untuk katalog dan Graf untuk rekomendasi. Jelaskan bagaimana menambahkan fitur PENCARIAN MULTI-KRITERIA (misalnya: buku kategori 'Teknik' dengan status TERSEDIA, diurutkan berdasarkan judul). Apakah BST cocok untuk query ini? Usulkan penambahan struktur data (misalnya secondary index) dan analisis kompleksitasnya.

Jawaban: Mencari buku kategori Teknik dengan status TERSEDIA dan diurutkan berdasarkan judul, BST kurang cocok untuk query multi-kriteria karena BST pada sistem ini hanya menggunakan ISBN sebagai key utama. Jika ingin mencari: kategori, status, judul, maka sistem harus melakukan traversal seluruh node. Kompleksitasnya menjadi: $O(n)$ Solusi Secondary Index Dapat ditambahkan struktur data tambahan seperti: dictionary kategori, dictionary status, sorted list judul Contoh: kategori['Teknik'] langsung mengakses daftar buku Teknik tanpa traversal seluruh BST. Kompleksitas Baru Pencarian kategori menggunakan hash map: $O(1)$ Sedangkan pengurutan judul: $O(n \log n)$

BAB VII

KESIMPULAN

7.1 Ringkasan Pencapaian

Pada proyek *Smart Library Management System* ini, sistem berhasil dibuat menggunakan beberapa struktur data seperti Binary Search Tree (BST), Queue, Stack, dan Graph. BST dipakai untuk menyimpan data buku berdasarkan ISBN supaya proses pencarian buku bisa lebih cepat dibanding pencarian biasa satu per satu. Queue digunakan untuk mengatur antrian pemesanan buku dengan metode FIFO (*First In First Out*), sedangkan Stack digunakan untuk menyimpan riwayat transaksi dan fitur *undo* transaksi dengan konsep LIFO (*Last In First Out*). Selain itu, Graph digunakan untuk membuat fitur rekomendasi buku berdasarkan hubungan peminjaman antar buku.

Sistem yang dibuat sudah dapat menjalankan fitur utama perpustakaan digital, seperti pencarian buku, peminjaman, pengembalian, pemesanan buku, pengelolaan antrian, *undo* transaksi, dan rekomendasi buku. Dari implementasi ini dapat dilihat bahwa penggunaan struktur data yang sesuai dapat membantu meningkatkan efisiensi pengolahan data dan membuat pengelolaan sistem perpustakaan menjadi lebih mudah dan terstruktur.

7.2 Refleksi Trade-Off Desain

Dalam perancangan sistem, pemilihan struktur data memiliki kelebihan dan kekurangan masing-masing. BST dipilih karena pencarian data lebih cepat pada kondisi normal dengan kompleksitas mendekati $O(\log n)$, tetapi jika data dimasukkan secara berurutan maka BST menjadi tidak seimbang dan proses pencarian bisa melambat menjadi $O(n)$.

Queue digunakan untuk mengatur antrian pemesanan buku karena cocok dengan metode FIFO sehingga urutan pemesan tetap terjaga. Stack digunakan untuk menyimpan riwayat transaksi dan fitur *undo* karena proses push dan pop dapat dilakukan dengan cepat dalam $O(1)$. Selain itu, Graph digunakan untuk fitur rekomendasi buku berdasarkan hubungan peminjaman, namun membutuhkan memori tambahan dan proses BFS untuk pencarian rekomendasi.

7.3 Saran Pengembangan

Pengembangan sistem ke depannya dapat dilakukan agar sistem menjadi lebih optimal dan lebih mendekati penggunaan di dunia nyata. BST biasa dapat diganti dengan AVL Tree atau Red-Black Tree supaya proses pencarian data tetap cepat dan stabil meskipun jumlah data sangat banyak.

Selain itu, sistem juga dapat ditambahkan database seperti MySQL atau SQLite agar data tidak hilang saat program ditutup. Tampilan sistem bisa dikembangkan menjadi berbasis GUI atau web supaya lebih menarik dan mudah digunakan. Fitur login admin dan anggota juga dapat ditambahkan untuk meningkatkan keamanan sistem. Pada bagian rekomendasi buku, metode yang lebih canggih seperti pembobotan frekuensi peminjaman atau machine learning dapat digunakan agar hasil rekomendasi menjadi lebih akurat.

LAMPIRAN

Pembagian Tugas Kelompok

- Maulana Fatihah Rizki_25051030075: Implementasi BST (Katalog Buku) & Fondasi Data Structures (Linked List).
- Nasya Afina Suharta_25051030068: Implementasi Sorting (Laporan Bulanan) & Analisis Performa (Runtime).
- Faisal Abdullah Hamood Hazaea_25051030061: Implementasi Stack (Riwayat Transaksi & Undo).
- Fairuz Addzikri Hakim_25051030063: Implementasi Queue (Antrian Pemesanan).
- Erlangga Supranoto Anggota_25051030041: Implementasi Graph (Sistem Rekomendasi Ko-pinjam).

Tabel Runtime

Struktur Data/Algoritma	Operasi	Kompleksitas Runtime
BST	Insert Buku	$O(\log n)$ average
BST	Search Buku	$O(\log n)$ average
BST	Travesal Inorder	$O(n)$
Queue	Enqueue Antrian	$O(1)$
Queue	Dequeue Antrian	$O(1)$
Stack	Push Riwayat	$O(1)$
Stack	Pop Undo	$O(1)$
Graph BFS	Rekomendasi Buku	$O(V+E)$
Shell Sort	Sorting Transaksi	$O(n^{1,5})$
Merge Sort	Sorting Buku Populer	$O(n \log n)$

Log AI

Aktivitas Penggunaan AI	Keterangan
Pembuatan struktur dasar program	Membantu menyusun kerangka program Python
Debugging kode	Membantu mencari dan memperbaiki error program
Penjelasan struktur data	Membantu memahami BST, Queue, Stack, dan Graph
Pembuatan dokumentasi laporan	Membantu menyusun isi BAB 1 sampai BAB 7
Analisis kompleksitas	Membantu menghitung Big-O runtime
Pembuatan diagram sistem	Membantu membuat diagram arsitektur sistem